RTSR 仿真过程与 trace 说明

一次完整的小规模仿真（3 个 UE / 100 MHz / 20 s）的执行过程、控制台输出与 FlowMonitor trace 解析。

一、这次仿真做了什么

```
cd ~/code/ns-allinone-3.39/ns-3.39
./ns3 run "cttc-nr-traffic-3gpp-xr --arUeNum=1 --vrUeNum=1 --cgUeNum=1 \
           --bandwidth=100e6 --numerology=1 --appDuration=20000"
```

实际耗时：57 s（3 UE，20 s 仿真）

步骤	内容	代码位置
① **拓扑**	gNB 布站 + UE 撒点（就近接入）	rtsr-simulation.cc
② **信道**	4 GHz / 100 MHz / 30 kHz，所有 gNB 同频	NrHelper 配置
③ **QoS**	视频 → 5QI 80；SR 模型 → 5QI 9	ActivateDedicatedEpsBearer
④ **业务**	XrTrafficMixerHelper 按 3GPP XR 模型生成 burst	.cc:120
⑤ **模型流**	UdpClient 530 包 × 1024 B，每 6 s 一次，端口 5000	.cc:72-84
⑥ **运行**	20 s 仿真，逐 slot 调度	Simulator::Run()
⑦ **统计**	FlowMonitor → 控制台 + XML trace	CheckForLostPackets / SerializeToXmlFile

二、控制台输出（本文数据均取自该次运行）

仿真共产生 14 条流：5 条视频流（AR 三条子流 + VR + CG）+ 9 条 SR 模型流。下表为终端输出的完整明细，与运行日志逐行对应。

流	端口	类型	tx 包	rx 包	丢包	流量	时延	抖动	吞吐
1	1121	AR 视频	4900	4900	0	0.60	1.679	0.012	0.3
2	1122	AR 视频	1176	1176	0	11.71	1.617	0.170	5.0
3	1123	AR 视频	4900	4900	0	1.90	1.679	0.012	0.8
4	1131	VR 视频	1177	1177	0	11.72	1.623	0.167	5.0
5	1141	CG 视频	1173	1173	0	11.68	1.631	0.171	5.0
6	5000	SR 模型	530	490	40	0.53	21.145	0.076	107.3
7	5000	SR 模型	530	490	40	0.53	18.674	0.074	109.8
8	5000	SR 模型	530	490	40	0.53	18.700	0.071	112.8
9	5000	SR 模型	530	479	51	0.53	24.343	0.077	105.5

10	5000	SR 模型	530	498	32	0.53	17.095	0.067	120.0
11	5000	SR 模型	530	494	36	0.53	17.594	0.068	118.6
12	5000	SR 模型	530	483	47	0.53	21.338	0.077	104.9
13	5000	SR 模型	530	475	55	0.53	19.941	0.071	109.3
14	5000	SR 模型	530	513	17	0.53	18.227	0.067	119.7

```
Mean flow throughput: 73.1474Mbps
Mean flow delay: 13.2347 ms
```

说明一：SR 模型每次更新都新建一个 UdpClient，因此一次更新就是一条独立流（3 个 UE × 3 次更新 = 9 条），这也是流量账用「发送量 ÷ 530 KiB」反推更新次数的原因。

说明二：ns-3 使用固定随机种子，重复运行该命令得到的输出逐位一致（已用两次独立运行核对：Tx Bytes、平均时延、抖动、汇总行完全相同），因此本文数据可复现。

三、生成的 trace 文件

文件	大小	内容
rtsr-run.log	4.2 KB	控制台输出全文（上表 14 条流）
rtsr-flowmon.xml	61 KB	FlowMonitor trace：每流计数器 + 每包时延直方图
README.md	3.7 KB	结构说明与解析注意事项

```
<FlowMonitor>
  <FlowStats>
    <Flow flowId="1" txBytes="627200" rxBytes="627200"
          txPackets="4900" rxPackets="4900" lostPackets="0"
          delaySum="+8.23e+09ns" jitterSum="+5.85e+07ns">
      <delayHistogram nBins="...">
        <bin index="16" start="0.0016" width="0.0001" count="4715" />
        ...
      </delayHistogram>
      <jitterHistogram>...</jitterHistogram>
      <packetSizeHistogram>...</packetSizeHistogram>
    </Flow>
  </FlowStats>
  <Ipv4FlowClassifier> ... </Ipv4FlowClassifier>
  <FlowProbes> ... </FlowProbes>
</FlowMonitor>
```

关键点：控制台只打印每流均值，XML 里才有逐包时延直方图（bin 宽 0.1 ms），因此可以从 trace 直接计算「超过 PDB 的包比例」，不必另写统计模块。

直方图	单位	用途
delayHistogram	秒（0.1 ms/bin）	计算 PDB 超限比例

jitterHistogram	秒（1 ms/bin）	抖动分布
packetSizeHistogram	字节（20 B/bin）	包长分布

四、trace 分析结果（PDB = 10 ms）

```
=== aggregate by class (PDB = 10 ms) ===
SR model  tx=4770 pkts  delivery=92.5%  delay>PDB=76.4%  bytes=4.79 MB
XR video  tx=13326 pkts delivery=100.0% delay>PDB= 0.0%  bytes=37.61 MB
```

结论：同一条链路、同一时刻，视频 100% 投递且没有一个包超过 10 ms；SR 模型流投递率 92.5%，且 76.4% 的包超过 10 ms。分层 QoS（视频高优先级、模型低优先级）在包级就已明显生效。

五、与论文 FSR 的关系

指标	论文	本次 trace 能给出的
FSR（帧级，10 ms PDB）	成功帧数 / 总帧数	尚不能（需按 burst 聚合成帧）
包级 PDB 满足率	论文未给	视频 100%，模型 23.6%

论文的 FSR 是应用层视频帧成功率：一帧在 BS 缓冲等待超过 10 ms 即整帧丢弃，模型更新的 PDB 为 50 ms。由于 5G-LENA 生成的包内没有帧号（XR 头被注释掉），要计算帧级 FSR 必须按 burst 边界把包重新聚合成帧，这一步尚未实现。

六、复现与文件位置

```
# 重新生成
cd ~/code/ns-allinone-3.39/ns-3.39 && ./ns3 run "cttc-nr-traffic-3gpp-xr ..."

# 分析 trace
python3 analyze_trace.py results/raw/traces/rtsr-flowmon.xml --pdb-ms 10
```

路径	说明
RTSR/rtsr-simulation.cc	新增 SerializeToXmlFile，导出 trace
RTSR/analyze_trace.py	解析 XML，输出每流与分类的 PDB 比例
RTSR/results/raw/traces/	本次运行的 log、xml 与说明